

LightPool: A NVMe-oF-based High-performance and Lightweight Storage Pool Architecture for Cloud-Native Distributed Database

Jiexiong Xu^{†*}, Yiquan Chen^{†*}, Yijing Wang^{*}, Wenhui Shi[‡], Guoju Fang^{*}, Yi Chen[†]
 Huasheng Liao[‡], Yang Wang[‡], Hai Lin[‡], Zhen Jin^{†*}, Qiang Liu^{*}, Wenzhi Chen[†]
 Zhejiang University[†], Alibaba Group^{*}, AntGroup[‡]
 Email: jiexiong.xjx@alibaba-inc.com, zy.zhengyi@alibaba-inc.com, chenwz@zju.edu.cn

Abstract—Emerging cloud-native distributed databases rely on local NVMe SSDs to provide high-performance and high-available data services to many cloud applications. However, the database clusters suffer from low utilization of local storage because of the imbalance between CPU and storage capacities within each node. For instance, the OceanBase distributed database cluster, with hundreds of PB local storage capacity, only utilizes around 40% of its local storage. Although disaggregated storage (EBS) can enhance storage utilization by provisioning the CPU and storage independently on demand, they suffer from performance bottlenecks and high costs.

In this paper, we propose LightPool, a high-performance and lightweight storage pool architecture large-scale deployed in the OceanBase clusters, enhancing storage resource utilization. The key idea of LightPool is aggregating cluster storage into a storage pool and enabling unified management. In particular, LightPool adopts NVMe-oF to enable high-performance storage resource sharing among cluster nodes and integrate the storage pool with Kubernetes to achieve flexible management and allocation of storage resources. Furthermore, we design the hot-upgrade and hot-migration mechanisms to enhance the availability of LightPool.

We have deployed LightPool on over 8500 nodes in production clusters. Statistics show that LightPool can improve storage resource utilization from about 40% to 65%. Experimental results show that the extra latency from LightPool is only about 2.1 μ s compared to local storage. Compared to OpenEBS, LightPool enhances bandwidth up to 190.9% in microbenchmarks and throughput up to 6.9% in real-world applications. LightPool is the best practice to deploy NVMe-oF (NVMe/TCP) in the production environment. We also discuss important lessons and experiences learned from the development of LightPool.

I. INTRODUCTION

Cloud-native distributed databases are widely deployed in cloud data centers, such as OceanBase [32], PloarDB-X [7], [12], MongoDB [30], Cassandra [16], offering high-performance and high-available database service to cloud applications. These databases adopt a distributed architecture that consists of multiple container instances to process application requests concurrently and can be flexibly scheduled by the container orchestration framework such as Kubernetes [1]. Compared to traditional databases, cloud-native distributed databases have superior scalability, availability, and fault tolerance.

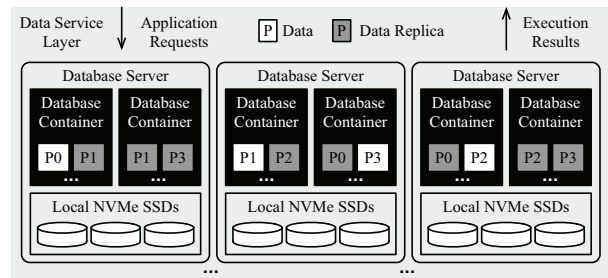


Fig. 1: The cloud-native distributed database architecture utilizing local NVMe storage in the production clusters.

To provide high-performance and reliable data services, cloud-native distributed databases have the following storage features.

- **Feature 1. High performance requirement.** These cloud-native distributed databases need to respond immediately to application requests and read/write large amounts of data, which are data-intensive, requiring high I/O performance with high throughput and low latency.
- **Feature 2. High capacity requirement.** Meanwhile, they also demand a large storage capacity to accommodate massive application data. For example, in OceanBase clusters, vast NVMe SSDs with hundreds of PB capacities have been deployed. Besides, each database container requests several to tens of TB of storage capacity based on application requirements.
- **Feature 3. Database-level data replicas.** They do not necessitate the storage system to guarantee data reliability since they support multiple data replicas at the database level (multiple data replicas of P0-P3 are shown in Fig. 1).

Mainstream cloud-native distributed database clusters adopt local NVMe SSDs directly attached to each node for data storage, as the local NVMe SSDs can adequately meet the above features. However, existing cloud-native distributed database clusters suffer from the **underutilization of local storage resources**. In the production clusters, the CPU, memory, and storage are often imbalanced within a node. Since each cluster node has fixed hardware resources and

the database containers have various CPU/Memory/Storage resources requirements, it is difficult to determine a machine configuration with the perfect balance between CPU, memory, and storage resources for all workloads [14], [15], [22], [34], resulting in underutilization of storage resources. Furthermore, the loads and specifications of each database instance are always changing rapidly. The fixed hardware configurations cannot be guaranteed to be always in balance with software requirements, exacerbating the underutilization of resources. For instance, our statistics show that local storage utilization of the OceanBase clusters with hundreds of PB local storage is only around 40%.

Many disaggregated storage solutions [4], [15], [21], [34] are proposed to enhance resource utilization. However, the disaggregated storage is unsuitable for the distributed clusters due to its performance bottlenecks and high cost. For performance, applications access the disaggregated storage through the cross-cluster network, which can easily become the bandwidth bottleneck and not satisfy the high-performance requirement (**Feature 1**). For the cost, disaggregated storage solutions are more expensive than local storage due to dedicated storage servers and network facilities. Cloud-native distributed databases store hundreds of PB data (**Feature 2**), resulting in a great increase in total cost of ownership (TCO). Furthermore, disaggregated storage solutions maintain multiple data replicas to ensure reliability, which becomes redundant for cloud-native distributed databases (**Feature 3**). This redundancy amplifies the cost difference between disaggregated storage and local storage solutions.

Even though the OpenEBS [33] is integrated into the Kubernetes framework [1] and can provide storage to cloud-native distributed databases, it cannot solve the storage resources underutilization challenge. The OpenEBS focuses on data reliability with multiple data replicas. Particularly, OpenEBS must allocate at least one data replica in local node [25], which cannot solve the storage resources underutilization challenge faced by cloud-native distributed database clusters.

In this paper, we propose LightPool, a novel lightweight and high-performance storage pool architecture for cloud-native distributed database clusters, enabling cross-node sharing of cluster storage resources to improve resource utilization. The key idea of LightPool is to aggregate cluster storage into a storage pool and adopt NVMe-oF to share high-performance NVMe storage among cluster nodes. We choose NVMe-oF for its excellent performance and stability and integrate it with the Kubernetes scheduling framework for efficient management and allocation of the storage resources. LightPool enhances cluster storage resource utilization and provides high-performance storage services comparable to local storage. LightPool consists of two parts: (1) LightPool-Controller is responsible for storage resources management and allocation. (2) LightPool-Initiator and LightPool-Engine achieve high-performance I/O paths for local and remote storage access. To achieve near-native storage performance, LightPool adopts a lightweight I/O stack and zero-copy mechanism for local access. Furthermore, we design the hot-upgrade and hot-

migration mechanism to enhance the availability of LightPool for the deployment in the production clusters.

We implemented and deployed LightPool on over 8500 nodes of production clusters, providing storage services to OceanBase [39] and achieving storage resource utilization enhancement from about 40% to 65%. LightPool is the first best practice to large-scale deploy NVMe-oF in the Alibaba cloud data centers. The deployment results show that LightPool can provide performance close to native NVMe disks that only incurs at least 2.1 μ s extra latency and achieve performance improvement up to 190.9% compared to OpenEBS.

The main contributions of this paper are as follows:

- **Novel Storage Pool Architecture.** We propose LightPool, a novel NVMe-oF-based high-performance and lightweight storage pool architecture, which enhances storage resource utilization by aggregating the cluster storage resources into the pool and sharing them across nodes through the NVMe-oF.
- **High-Performance Storage Stack.** LightPool utilizes the high-performance NVMe-oF protocol for storage access and a lightweight I/O stack, achieving near-native performance. To optimize the local access in the storage pool, we propose a zero-copy mechanism to transfer I/O requests efficiently and eliminate redundant data copies in the local I/O path.
- **Large-scale Deployment and Evaluation.** LightPool is the best practice to large-scale deploy NVMe-oF in production clusters. LightPool has been deployed on over 8500 nodes and enhances storage utilization from 40% to 65%. We also shared our lessons and experience from development of LightPool.

II. BACKGROUND

A. NVMe and NVMe over Fabrics

NVM Express (NVMe) [31] is a high-performance protocol for accessing local non-volatile memory devices over PCIe [3] bus. Due to their vastly superior performance in terms of bandwidth and latency compared to HDDs and SATA-SSDs, NVMe SSDs are widely chosen in cloud data centers to cater to the ever-increasing demands of I/O-intensive applications.

To enable access NVMe devices on the remote server over different network fabrics, the NVMe Working Group proposed the NVMe over Fabrics (NVMe-oF) [31] as an extension of the NVMe. NVMe-oF enables access to high-throughput and low-latency remote NVMe devices, achieving performance close to locally attached NVMe SSDs. It outperforms existing remote storage protocols such as iSCSI in both performance and CPU resource consumption [17].

B. Kubernetes and Container Storage

Kubernetes [1] is a popular container orchestrator which is large-scale deployed in cloud data centers. A typical Kubernetes cluster is shown in Fig. 2. It contains control plane nodes and worker nodes. Pods are the smallest workload unit in Kubernetes, which contains one or more containers running on the control plane and worker nodes. Specifically, core

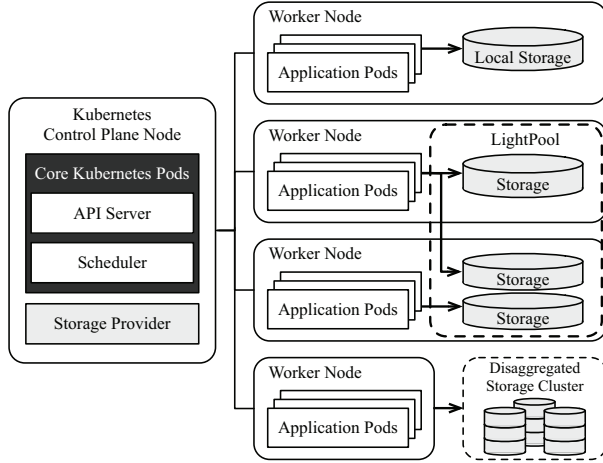


Fig. 2: The basic kubernetes cluster with different storage schemes: local storage, LightPool, and remote disaggregated storage.

Kubernetes components run on the control plane nodes, while user workloads run on the worker nodes. The core components are composed of (1) the API server, which is responsible for managing the state of the Kubernetes cluster and exposes HTTP endpoints for accessing the state, and (2) the scheduler, which assigns pods to nodes.

The storage provider in the Kubernetes cluster provides persistent storage to the pods in the form of *volumes*. Common storage options include *local storage*, *OpenEBS* and *storage disaggregation*. In the local storage scheme, user pods directly access storage devices attached to the same worker node through the PCIe bus, achieving high throughput and low latency. Existing cloud-native solution, OpenEBS [33] (Mayastor), aims at providing highly reliable cloud-native storage with multiple data replica and adopts NVMe-oF to store the data replicas on other nodes. By contrast, for storage disaggregation, user pods access storage devices in the remote dedicated storage cluster through the network.

III. MOTIVATION

We reviewed the state-of-the-art storage solutions for the cloud-native distributed database clusters. However, these solutions have several limitations regarding resource utilization, cost, and performance.

A. Low Resource Utilization and Inflexibility of Local Storage

The cloud-native distributed database clusters adopt local storage for its high bandwidth performance and large capacity with low cost. For local storage, NVMe SSDs are directly attached to the physical server and are accessed through the PCIe system bus. However, using the local storage scheme faces challenges of storage resource underutilization and inflexibility.

Storage resource underutilization. The local storage in the cloud-native distributed database clusters suffer from low utilization. The cluster nodes have fixed hardware configurations

and run containers requiring various CPU/Memory/Storage resources, which results in many nodes exhaust CPU and memory resources but have storage resources remaining.

For instance, the typical node configurations in OceanBase clusters are 82/104 CPU cores and 12*3.84TB Intel P4510 NVMe SSDs, while there are 2676 container configurations in production. Some of these containers consume massive CPU resources and a small amount of storage (e.g., 104 CPU cores, 400GB memory, and 2.85TB storage), causing a node run out of CPU cores and remain 93.8% storage resources. Based on statistics, the average utilization of local storage with hundreds of PB capacity in OceanBase production clusters is only about 40%, resulting in huge resource waste and TCO increase.

As a result, low utilization of local storage resources is a common challenge in cloud data centers. We want to enhance resource utilization by aggregating local storage into pools and sharing among cluster nodes to reduce storage costs.

B. Disadvantages of Disaggregated Storage

Mainstream cloud vendors offer highly scalable and reliable elastic block store (EBS) [4], [11] as commercial disaggregated storage solutions, which is built on dedicated storage clusters. Nevertheless, the EBS suffer from bandwidth performance bottleneck and high cost.

Bandwidth performance bottleneck. Applications access the EBS through the inter-cluster network, which easily becomes the bandwidth bottleneck for the storage services. The disaggregated storage solutions typically use a multi-tier switch architecture, where the network bandwidth of the aggregation switch is typically a few tenths of the bandwidth of top-of-rack switches. Therefore, the bandwidth of the aggregation switch in the cluster will limit the whole cluster bandwidth. As a result, the bandwidth performance provided by the disaggregated storage solutions is far from the bandwidth requirements of cloud-native distributed databases. In contrast, the local storage can offer scalable bandwidth performance with the same capacity of NVMe SSDs.

High cost. The EBS solutions have to deal with high cost due to the additional storage servers and network infrastructure, as well as redundant data replicas. Providing EBS to the cloud-native distributed database clusters requires building dedicated storage clusters and network facilities, resulting in a substantial increase in TCO for the same storage capacity. Meanwhile, the EBS solutions keep multiple data replicas to ensure reliability, which is redundant for database-level data reliability and amplifies the cost increase.

C. Disadvantages of Cloud Native Storage

Storage resources underutilization. OpenEBS inevitably requires one local data replica on the application node [25]. Even using replica volume, OpenBES requires that a data replica must be stored locally. Therefore, it cannot enhance the cluster storage utilization through sharing among cluster nodes.

Performance degradation. OpenEBS also suffers severe performance degradation due to the overhead from complex

software stacks. OpenEBS focuses on system availability and data redundancy rather than high-performance storage. Therefore, these considerations make their software stacks thick and lead to performance overhead. As mentioned, cloud-native distributed databases do not require multiple data replicas at the storage system level. This motivates us to design a lightweight storage pool architecture that focuses on high storage performance close to local storage with no data replica.

IV. DESIGN AND IMPLEMENTATION OF LIGHTPOOL

To provide high-performance storage services to cloud-native distributed databases, cloud vendors have strong demands for low cost, high resource utilization, and high-performance storage architecture. To this end, we set the following design goals to resolve local storage challenges.

Low cost: The storage scheme in production clusters should be low cost which does not need additional dedicated storage clusters.

High storage utilization: It should enable storage resources sharing among cluster nodes, achieving high storage resources utilization and reduce TCO.

High performance: It should provide throughput and latency performance close to native performance. The software stack of the storage architecture should be lightweight to decrease overhead.

High availability: It should provide high-available storage services to the application containers. Specifically, the container I/O should not be interrupted by component upgrades and data migration.

A. Design Choice

We have made many design choices to better provide high-performance and low-cost storage to cloud-native distributed databases.

Storage protocol choice. LightPool adopts the NVMe over Fabric as the remote storage protocol to share storage resources across nodes. NVMe-oF is the best choice for delivering a high-performance storage pool. Compared to iSCSI, NVMe-oF can fully exploit the inherent parallelism and multiple queues of the emerging NVMe devices. In particular, iSCSI suffers from up to 40% performance loss and consumes 30% more CPU resources for additional operations, such as protocol conversion, when accessing NVMe SSDs [17]. As reported by previous work [17], NVMe-oF can provide performance close to that of locally attached NVMe SSDs. Therefore, adopting NVMe-oF can minimize the overhead when accessing storage resources in our storage pool to achieve high performance.

Fabric choice. We choose TCP rather than RDMA as the network protocol for NVMe-oF. In cloud data centers, TCP is stable and does not require hardware support, so it is deployed on a large scale. In contrast, RDMA has deployment scale limitations and requires hardware support, which cannot be deployed on all machines in all OceanBase clusters. Therefore, adopting TCP can enable LightPool large-scale deployment in existing clusters. LightPool will support RDMA in the future as the hardware evolves.

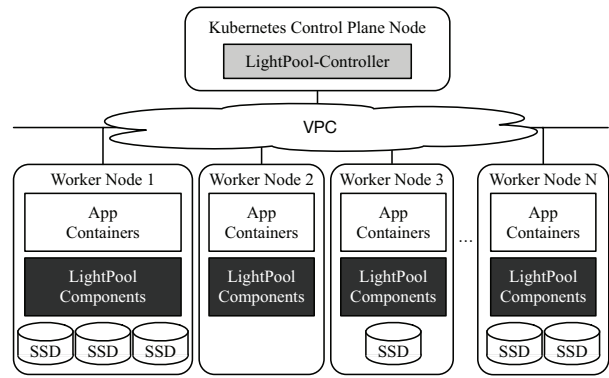


Fig. 3: LightPool cluster architecture overview.

No data replica. LightPool implements no data replica at the storage system level rather than multiple replicas as the other distributed storage schemes. The cloud-native distributed databases that use the LightPool typically use multiple data replicas at the database level, meaning that the application stores multiple replicas in the storage system. At the same time, unnecessary data redundancy can lead to a further increase in storage costs. On the other hand, multiple replicas in the storage system can lead to a more complex software stack design and increased performance loss. As a result, adopting no data replica scheme can reduce storage costs and performance loss.

B. Overall Architecture

LightPool is designed to aggregate storage resources into a storage pool, enabling the dynamic allocation of resources to any node in the cluster. It enhances storage resource utilization. The overall architecture of LightPool is illustrated in Fig. 3, which showcases the LightPool architecture. The LightPool cluster comprises two types of nodes: (1) Kubernetes control plane node, which runs the LightPool-Controller and is responsible for managing all storage resources in the cluster and allocating appropriate storage resources for the worker nodes when requested. (2) Worker nodes that run application containers and LightPool Components. Depending on the machine type, some worker nodes are equipped with NVMe SSDs (worker node 1, 3, N), while some only run applications (pods) without attached storage devices (worker node 2). Each worker node can access the storage resources on itself or other worker nodes, which are allocated by the LightPool-Controller. Fig. 4 demonstrates LightPool components in detail.

Components on the Kubernetes control plane node. LightPool-Controller on the Kubernetes control plane node is the core component of LightPool. It maintains a global view of all storage resources in the cluster, and it allocates storage volumes from the storage pool for application pods. LightPool-Controller is composed of Storage Scheduler and Volume Manager. When the Storage Scheduler identifies a storage allocation request, it selects a target worker node with sufficient free storage space in the cluster based on

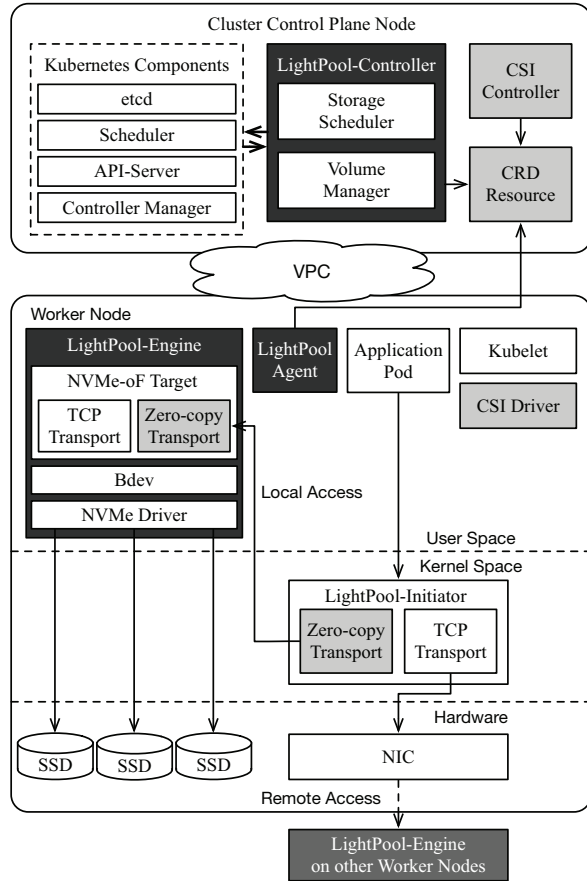


Fig. 4: LightPool key components.

the customized scheduling strategy. Subsequently, the Volume Manager notifies the target worker node to create a free volume with the specified capacity. Finally, the Volume Manager stores the allocated storage information (e.g., target node IP address, port, volume) into the Kubernetes control plane node, which can be accessed by the worker node.

Components on the worker node. Each worker node in LightPool runs LightPool Elements, including LightPool-Engine, LightPool-Agent, CSI Driver, and LightPool-Initiator, as shown in Fig. 4. The LightPool-Engine manages the storage devices on each node. LightPool-Agent gets the storage information from the LightPool-Engine and reports it to the LightPool-Controller. The CSI Driver would notify LightPool-Initiator to connect to LightPool-Engine on the target worker node when the pod is scheduled to the node.

For high-performance storage, we choose NVMe-oF to access the storage pool with low latency and high throughput. To further reduce software overhead, we adopt a lightweight stack for LightPool. In particular, we optimize the software stack for accessing local storage through zero-copy mechanism, achieving near-native performance. We propose a novel transport called zero-copy for the NVMe-oF for local storage access. As a result, LightPool achieves extreme performance

Algorithm 1 Storage allocation algorithm of LightPool.

Stage 1: pre-filtering

```

1: for Node in Node_List do
2:   Selected_List.append(Nodes that meets the require-
   ments)
3: end for
4: return the Selected_List

```

Stage 2: selecting

```

1: for Node in Selected_List do
2:   if (PreferLocal && HostNode.ID == Node.ID) or (Pre-
   ferRemote && HostNode.ID != Node.ID) then
3:     Node.Score += 20
4:   end if
5:   Total = Node.Total, Free = Node.Free
6:   Used_Score = 100 * (Total - Free) / Total
7:   Node.Score += Used_Score
8: end for
9: return the Node with the maximum Score

```

that is close to local disks with minimal overhead.

C. LightPool Storage Resources Management

LightPool aggregates the local storage attached to each node into a storage pool to enhance the storage utilization of the cluster. Specifically, the LightPool-Controller running on the Kubernetes control plane node is responsible for managing and allocating storage resources for application pods in the whole clusters.

Storage aggregation. In the initial stage of the cluster, the LightPool-Agent on each worker node get the information of available storage resources from the LightPool-Engine through RPC. Then, the LightPool-Controller collects this information and stores them in a list, in which each entry holds information of a worker node.

Storage pool maintenance. LightPool-Controller maintains a node list, which contains the node and storage information of each cluster node. The storage information comprises the allocated volumes and free storage resources, which keep track of all persistent volumes (PV) assigned to the application pods and all available storage resources, respectively. In summary, the LightPool-Controller can efficiently manage the storage pool and schedule available storage resources to application pods through the node list.

Allocating strategy. When a new application pod is created, the LightPool-Controller on the Kubernetes control plane node receives the storage requirements and selects a worker node as the target. Then, the LightPool-Agent on the selected worker node allocates a volume with a certain amount of capacity and notifies the LightPool-Engine to wait for the connection.

The Storage Scheduler in the LightPool-Controller utilizes a two-stage allocating strategy, as demonstrated in Algorithm 1. In the pre-filtering stage, the Storage Scheduler traverses the Node List to identify all worker nodes that meet the storage requirements. Next, in the selecting stage, the Storage Scheduler scores the selected worker nodes based on whether

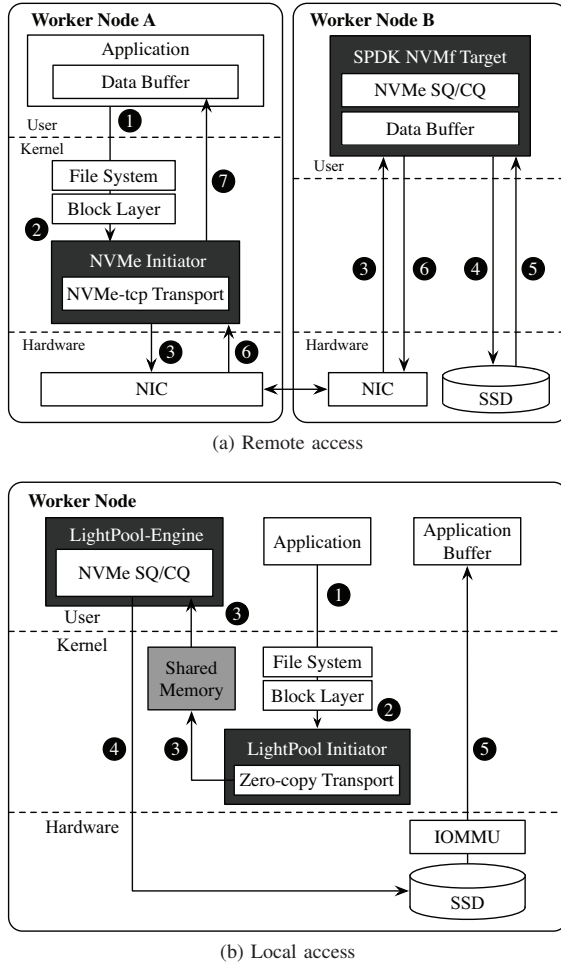


Fig. 5: The I/O and data path of LightPool

they meet the user's local/remote preference and the ratio of remaining available capacity. Finally, the worker node with the highest score is chosen as the target worker node. As a result, the Storage Scheduler prioritizes allocating storage resources from nodes with less free storage capacity to reduce fragmentation, which enables reserving enough space on a single node to satisfy large-capacity storage allocation requests in the future.

After determining the storage provider worker node, the LightPool-Agent on the selected worker node notifies LightPool-Engine to create a logical volume of specified capacity. Then, the LightPool-Initiator on the application worker node connects to the LightPool-Engine on the selected worker node through Kubernetes components (CSI Driver). Finally, the application pod can utilize storage resources on the selected worker node.

D. The I/O Path of LightPool

LightPool adopts a lightweight stack to reduce the overhead: a user-level LightPool-Engine and a kernel-level LightPool-Initiator. The LightPool-Engine directly accesses and manages

physical devices on each node. The LightPool-Initiator can connect to LightPool-Engine on the selected worker node, and presents the allocated storage resources as standard NVMe devices for application pods access.

The storage location is determined by the LightPool-Controller, which can be local or remote. The two types of the I/O path in LightPool are shown in Fig. 5. The key components involved in the I/O path contain the application pod, LightPool-Initiator, LightPool-Engine, and storage devices. In LightPool, applications can access the storage resources in the local and remote worker nodes through the NVMe-oF I/O stack.

Remote I/O path. Fig. 5a presents the remote I/O path in LightPool. The LightPool-Initiator presents standard NVMe block devices in the kernel to the application pods. ① Application pods read/write files in the file system mounted on the standard NVMe devices to access storage resources and are unaware of the location of physical disks. ② The LightPool-Initiator on the application worker node receives I/O requests issued by the applications and ③ forward it through TCP stack (NIC) to the remote user-level LightPool-Engines on the target worker node. ④ The LightPool-Engine would convert these requests into NVMe commands and notify physical storage devices to fetch these commands. ⑤ Next, the storage devices get the NVMe command and read/write the data to the data buffer in the LightPool-Engine. ⑥ Then, the data would be sent to the LightPool-Initiator through the TCP by the LightPool-Engine. ⑦ Finally, the data is copied to the application buffer.

Local I/O path with zero-copy mechanism. Fig. 5b demonstrates the local I/O path in that application pods access storage resources on the same worker node. We propose the DMA remapping by setting the IOMMU page table and replacing the address of the user buffer with IOVA when the I/O request reaches the Initiator. The steps ① and ② are the same as the remote I/O path. Originally, when accessing local disks, I/O requests and data were transferred between the LightPool-Initiator and the LightPool-Engine through tcp-loopback. However, this process involved two redundant data copies, causing performance degradation and CPU overhead. To eliminate the above problems, we propose a novel zero-copy transport for local access in LightPool. For I/O requests transmission, ③ the zero-copy transport adopts shared memory between the LightPool-Initiator and the LightPool-Engine. The LightPool-Initiator converts the I/O requests into NVMe commands and replaces the user buffer address with the IOVA, which is registered in the IOMMU page table. ④ the LightPool-Engine receives the NVMe commands, puts them in the submission queues, and notifies physical storage devices to fetch these commands. For data, ⑤, we propose a DMA remapping mechanism to enable the local storage devices can directly access the application buffer. The zero-copy transport adopts shared memory to eliminate redundant data copies between the user buffer and kernel, achieving near-native performance when accessing local storage resources.

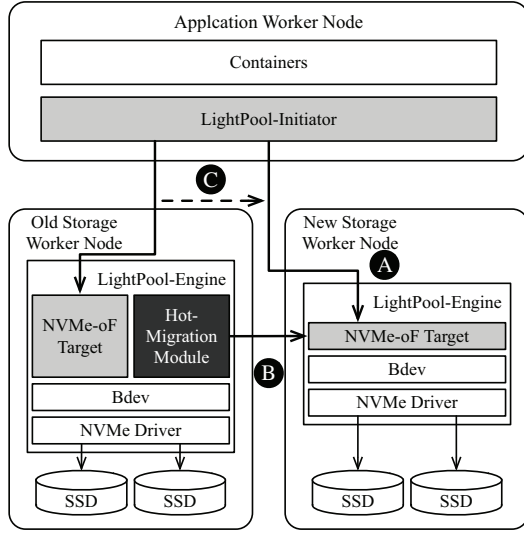


Fig. 6: The hot-migration process of LightPool.

E. High Availability of LightPool

Storage service availability is as essential as storage performance. To provide high-available storage services to the containers, we design the hot-upgrade and hot-migration mechanism for the LightPool, which can upgrade the LightPool-Engine and migrate the container data to other nodes without application awareness, respectively.

Hot-upgrade. The LightPool-Engine is one of the key components on the LightPool I/O path and requires regular upgrades to add new features and bug fixes. Therefore, we propose a hot-upgrade mechanism to ensure that the upgrade process of the LightPool-Engine don't interrupt application I/O.

When hot-upgrade starts, the LightPool-Engine will first enter the hot-upgrade state and call the `fork` system calls to create a child process which is the new version of the LightPool-Engine. This child process will wait until the parent process (the old version of the LightPool-Engine) finishes. Then, the parent process will stop receiving I/O requests and wait for requests in the process to complete. Next, it will close all subsystems and mark itself as exited. During this period, the I/O requests sent by the application will be blocked but will not raise errors. When the child process finds the parent process has exited, the child process starts to restore the subsystem based on the configuration file and re-establish the TCP connection with the initiator. Finally, the child process (the new version of the LightPool-Engine) starts to process I/O requests and finishes the hot-upgrade process.

Hot-migration. In production clusters, the administrator needs to migrate the application data to another node in many occasions, such as the storage-provider node has to be maintained, or a portion of storage capacity allocated to other nodes needs to be released for local applications. Therefore, we propose the hot-migration mechanism based on multipath-based dynamic I/O path switching to migrate the data to

TABLE I: Details of our experimental setup

Host	Description
CPU	Intel Xeon Platinum 8163 CPU
RAM	768GB DDR4
Host OS	CentOS 7.9.2009
Container image	CentOS 7
Kernel Version	5.15.0
SSD	2.0 TB Intel P4510 NVMe SSD

the new node without interrupting storage services. Fig. 6 demonstrates the steps to perform a data hot-migration that migrate the data from the old storage worker node to the new one.

A At the beginning of the hot-migration, the administrator first creates a target subsystem in the new storage worker node with the same NQN and uuid as the target subsystem on the old storage worker node. Then the LightPool-Initiator will connect to the new target subsystem through the multipath and set the new connection to inactive.

B The hot-migration module of the LightPool-Initiator on the old worker node starts copying data multiple rounds. In the first round, the hot-migration module copies all data. At the same time, it uses a bitmap to record whether the data corresponding to each LBA address has been modified (write or discard operations). In the following rounds, the hot-migration module copies the modified data recorded in the bitmap to the new storage worker node while using a new bitmap to record the modifications in the current round of copies. Finally, if the data we need to copy can be completed within 3-5s, the hot-migration module notifies the LightPool-Initiator to pause submitting application requests and perform the data copying.

C After the final round of data copying, the LightPool-Initiator will switch the I/O path to the target subsystem of the new storage worker node by dropping old connections and enabling new ones. Since the multipath reserves the `/dev/nvme0n1` used for application containers, the hot-migration process avoids interrupting storage service caused by the disappearance of the block device and remount operations.

As a result, during the whole hot-upgrade and hot-migration process, the user can read and write data as usual without interrupting storage service and I/O errors. We enhance the availability of LightPool through these features.

V. EVALUATION

This section evaluates the storage performance of LightPool and compares its I/O performance with other storage schemes through synthetic benchmarks and real-world applications running in the Kubernetes containers. For LightPool deployed in the production environment, we demonstrated the SQL performance of the OceanBase and its corresponding I/O performance.

A. Experimental Setup

System settings. Table I presents the detailed experimental settings for LightPool. We built a 4 nodes Kubernetes cluster,

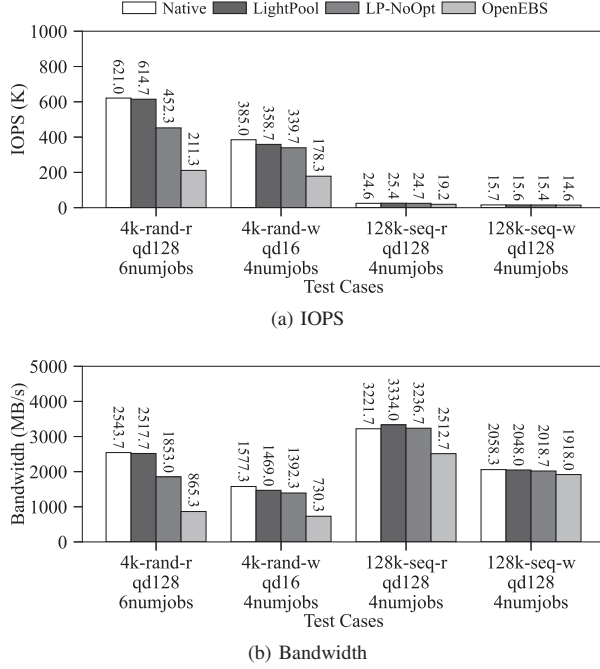


Fig. 7: The IOPS and bandwidth of FIO with 1 disk of native disks, LightPool, LightPool without zero-copy (LP-NoOpt), and OpenEBS.

including one control plane node and three worker nodes. We adopted the centos:7 docker image and installed evaluation applications to evaluate storage I/O performance. We also disabled hyperthreading and power-saving states for accurate performance measurements on the servers. The production data is obtained from the monitoring system of the OceanBase production cluster.

Baseline and compared methods. We took the performance of the native disks as the baseline and evaluated the LightPool and LightPool without the zero-copy mechanism when allocating local storage resources. We also compared LightPool with OpenEBS Mayastor [26] (using local storage and no data replica), the mainstream cloud-native storage solution. In particular, the Mayastor requires a minimum of three worker nodes in the cluster. However, it cannot only allocate remote storage resources to enhance cluster storage utilization.

Benchmarks and applications. We leveraged FIO [2] to generate synthetic I/O workloads. FIO is a powerful tool for testing the performance of storage devices, which has been widely used by industry and academia. Specifically, we took the libaio as the I/O engine and ran multiple test cases, including random and sequential read and write with various block sizes, queue depths, and the number of jobs. To evaluate LightPool in the real-world application, we adopted YCSB [13] to drive the RocksDB database. We also present the SQL and disk I/O performance of the OceanBase utilizing LightPool in the production cluster.

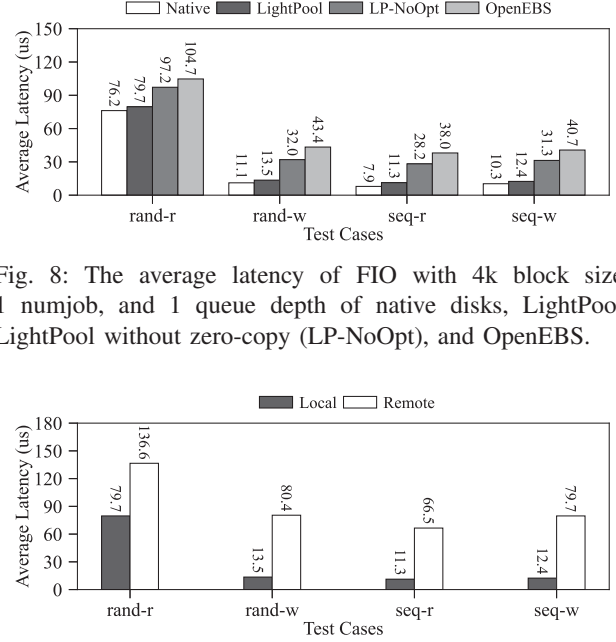


Fig. 8: The average latency of FIO with 4k block size, 1 numjob, and 1 queue depth of native disks, LightPool, LightPool without zero-copy (LP-NoOpt), and OpenEBS.

Fig. 9: The average latency results of FIO with 4k block size, 1 numjob, and 1 queue depth of local and remote storage of LightPool.

B. Performance

To evaluate the I/O performance in the centos container, we ran FIO and measured the IOPS, bandwidth, and average latency of the native disk, LightPool, LightPool without zero-copy mechanism, and OpenEBS. In both LightPool and OpenEBS, we allocated the local storage resources on the same node running the application containers. We allocated a 1200 GB persistent volume and attached the volume to the application containers. We utilized the FIO to read and write an 1000 GB file on the directory corresponding to the persistent volume mounted in the container to evaluate the I/O performance. We also evaluated the performance of local and remote storage resources allocated by LightPool.

IOPS and bandwidth performance. The IOPS and bandwidth performance of the native disk, LightPool, LightPool without zero-copy optimization (LP-NoOpt), and OpenEBS are demonstrated in Fig. 7. We observed that LightPool could achieve performance from 93.2% to 103.4% of the native disk, indicating the performance loss and the overhead of LightPool is minimal. However, Mayastor can only achieve 34.0% and 46.3% of native disks in random read (4k-rand-r-qd128-6numjobs) and write (4k-rand-w-qd16-4numjobs) test cases. In the best case (4k-rand-r-qd128-8numjobs), the performance of LightPool is 190.9% higher than that of OpenEBS. LightPool benefits from its lightweight software stack and achieves near-native storage performance, while OpenEBS suffers severe performance degradation. In addition, OpenEBS only supports local storage allocation on the same node where the applications run. As a result, LightPool can aggregate cluster storage

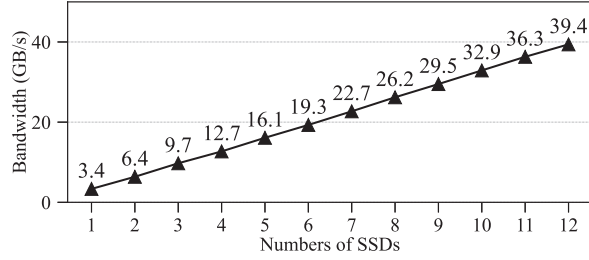


Fig. 10: Total bandwidth of LightPool in different numbers of SSDs.

resources to enhance cluster storage utilization. For the zero-copy mechanism, the bandwidth of LightPool outperforms up to 35.9% with that without zero-copy, indicating that zero-copy mechanism can effectively optimize the performance of accessing local storage resources.

Latency performance. Fig. 8 demonstrates that the average latency performance of LightPool is much better than OpenEBS and achieves near-native performance, indicating that the lightweight stack and zero-copy mechanism can provide latency close to native NVMe SSDs. We can observe that LightPool introduces only about 2.1 to 3.5 μ s extra latency over the native disk in both sequential and random read and write test cases. The zero-copy mechanism eliminates redundant data copies and bypasses the tcp-loopback in LightPool and significantly reduces up to 60.4% average latency (in the sequential write test case) when application pods access local storage devices. As a result, LightPool can provide high-performance storage services with minimal overhead.

LightPool local and remote performance. We also evaluated the average latency performance of local and remote storage resources allocated by LightPool. For the remote storage resources, we utilized the 2*25 Gbps network. Fig. 9 shows that remote storage resources only incurs the constant 55.2 - 67.3 μ s extra latency compared to the local. Meanwhile, for the I/O throughput, remote storage resources would be impacted by network bandwidth. Fig. 9

C. Scalability

For scalability, we evaluated the I/O performance of LightPool on 1-12 NVMe SSDs since our typical node is configured with 12 disks. We tested the total bandwidth in one application container.

Fig. 10 shows the result of LightPool in 128k-seq-r-qd128-4numjobs. We can see that the total bandwidth increases linearly with the number of SSDs. LightPool can saturate 12 SSDs with 39.4 GB/s bandwidth. Therefore, we can conclude that *LightPool can ensure promising scalability.*

D. Real-world Application

In this section, we selected RocksDB as real-world application to evaluate the performance of LightPool. We deployed native disks, OpenEBS, and LightPool to provide storage to the RocksDB application. We installed RocksDB-6.4.6 in the

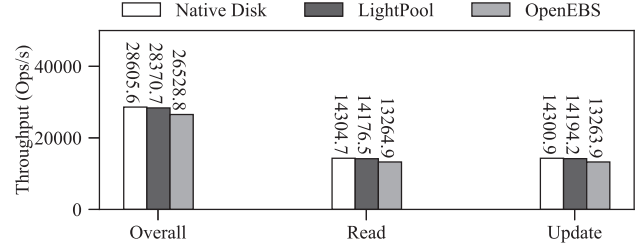


Fig. 11: The RocksDB throughput performance of native disk, LightPool, and OpenEBS.

TABLE II: Average latency result of RocksDB.

Operation	Native Disk	LightPool	OpenEBS
Read Lat (us)	27.6	28.0	30.5
Update Lat (us)	39.5	39.8	42.4

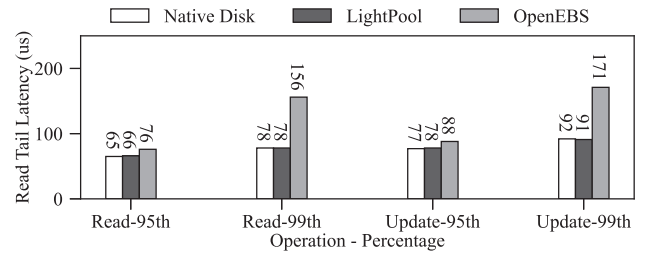


Fig. 12: The read and update operations tail latency performance of RocksDB.

Centos7 container and YCSB-0.15.0 to generate workloads. For workload parameters, we set the *recordcount* and *operationcount* to 10 million and set the *readproportion* and *updateproportion* to 0.5.

Throughput performance. As shown in Fig. 11, we can see that LightPool achieves near-native performance under real-world workloads. LightPool provides 99.2% overall throughput of the native disk with nearly no performance degradation. Specifically, the performance of read and update operations is 99.1% and 99.3% of native disks, respectively. Compared to OpenEBS, LightPool achieves 106.9% of overall throughput.

Latency performance. From the results in Table II and Fig. 12, we can conclude that LightPool can achieve near-native latency performance and outperform OpenEBS in the real-world application. LightPool can obtain the tail latency performance close to the native disk. Furthermore, the 99th latency of LightPool is only 50.0% and 53.2% of that of the OpenEBS Mayator in read and update operations, respectively. As a result, LightPool benefits from the lightweight software stack and achieves near-native storage performance.

E. Availability

In this section, we evaluated the availability of LightPool in terms of the I/O throughput when performing hot-upgrade and hot-migration, respectively. We performed hot-upgrade and hot-migration during the I/O operations.

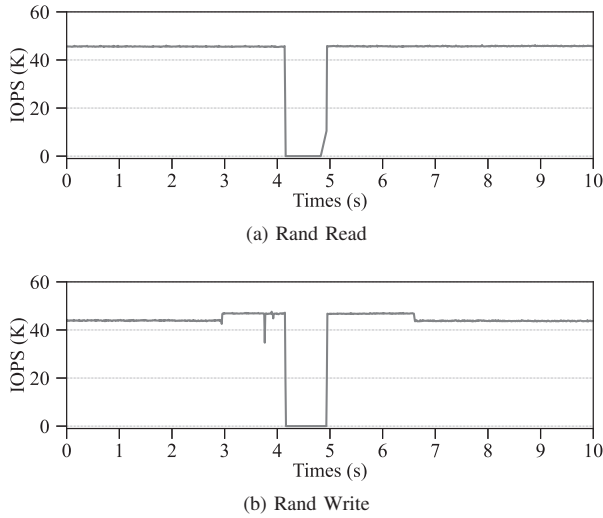


Fig. 13: The IOPS Performance of Application Container When Do Hot-upgrade of LightPool-Engine.

Hot-upgrade. To measure the I/O pause time caused by hot-upgrade, we ran the test case of random read/write with 4K block size, queue depth of 1, and 1 FIO job. Then we recorded the IOPS of FIO when performing hot-upgrade and presented the results in Fig. 13, which shows that the hot-upgrade of LightPool-Engine can be completed in only 1s.

In our production clusters, we also performed hot-upgrade test three times. Fig. 14 demonstrates that during hot-upgrade, the OceanBase SQL throughput is reduced by a maximum of 31.0%, and the latency is increased by a maximum of 1.49x, which lasts less than 10s and can be accepted by OceanBase instances.

Hot-migration. We also evaluate the data migration bandwidth when performing hot-migration. We allocated a 400GB volume and utilized FIO to keep writing the volume with 4KB block size. Then we started the hot-migration. The hot-migration finished within 7 rounds and the data migration bandwidth of each round is shown in Fig. 15. During the hot-migration, there is no interruption to FIO write I/O.

F. Performance Characteristics in Production Clusters

To demonstrate how LightPool works in the production environment, we deployed LightPool in production clusters over 8500 nodes to provide OceanBase with high-performance storage services, enhancing cluster storage utilization from about 40% to 65%.

LightPool provides high-performance storage services to the OceanBase clusters. Fig. 16 presents disk monitoring data of the local and remote storage resources allocated by LightPool in production clusters. Fig. 16a and Fig. 16d show that allocating remote or local resources has little impact on IOPS performance. For the await time performance in Fig. 16b and Fig. 16e, remote resources will be about 0.3 ms higher than local resources. Fig. 16c and Fig. 16f demonstrates the SQL Response Time (RT) of the OceanBase node deployed

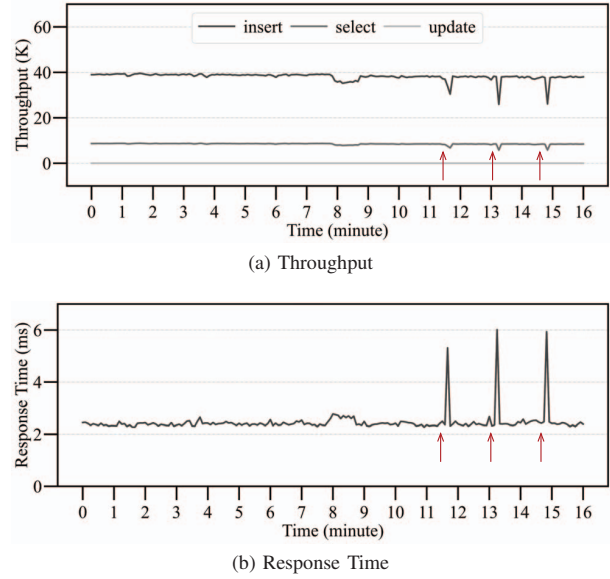


Fig. 14: The OceanBase SQL performance in production when do hot-upgrade of LightPool.

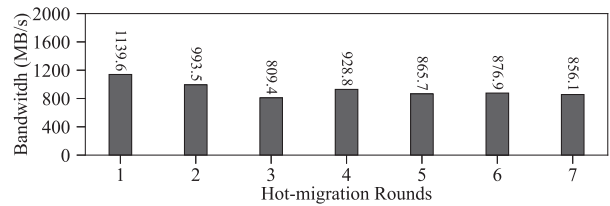


Fig. 15: The data migration bandwidth of LightPool when performing hot-migration.

with LightPool and QPS/TPS performance, respectively. The location of storage resources has little impact on the OceanBase SQL Response Time (RT) performance.

Consequently, LightPool can meet OceanBase's high-performance and low-cost storage requirements. Users can choose the location of storage resources (local or remote) based on the latency sensitivity.

VI. LESSONS AND EXPERIENCE

Consideration and experience on LightPool. We found that disaggregated storage solutions cannot solve everything. Through the NVMe-oF and the lightweight storage stack, we solve the resources utilization challenges commonly faced by cloud data centers by this efficient method. In contrast, disaggregated storage solutions are too heavy for cloud-native distributed databases and cause higher TCO. Since NVMe-oF is an industry standard that enables high-performance remote storage access while ensuring good compatibility, adopting standard NVMe-oF can accelerate the deployment of LightPool. As a result, LightPool is one of the best choices with low cost and high performance for cloud workloads that do

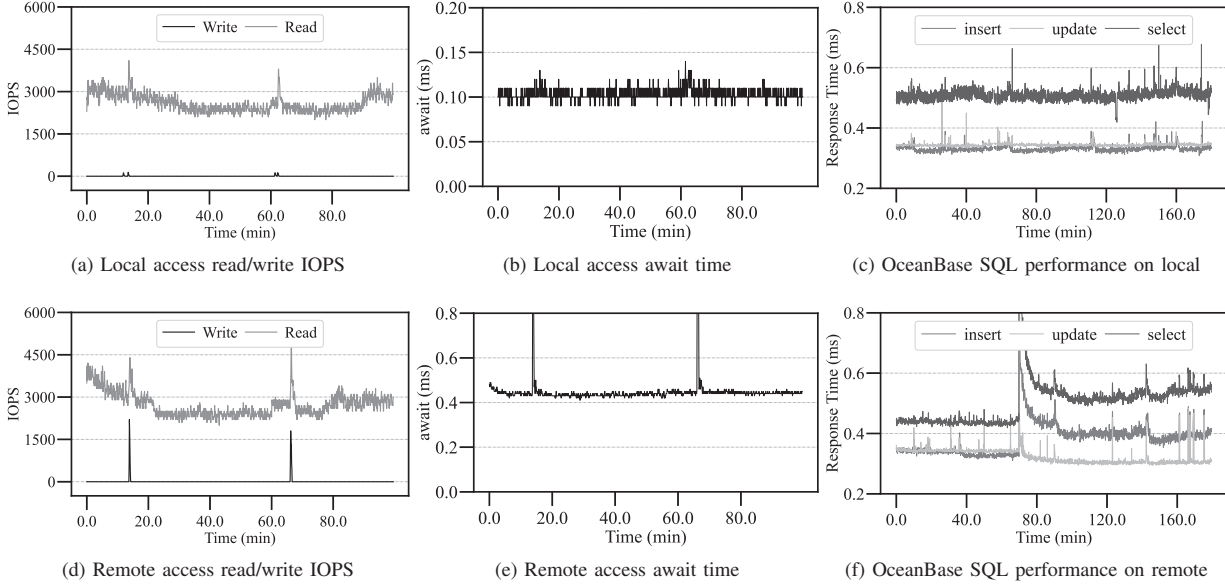


Fig. 16: Disk monitoring and OceanBase SQL performance of the LightPool local/remote storage in production clusters.

not require multiple data replicas but have high demands on storage bandwidth performance and capacity.

Many optimizations are needed for NVMe/TCP in the production with satisfactory performance. we met performance fluctuations that timeout requests may exist when the system load is low and the number of TCP connections is high. To solve this, we added barrier in the `qdisc_run_end()` function and commit the patch to the community, which has been merged into the Kernel 5.19. Meanwhile, we found performance overheads from unnecessary multiple data copies when accessing local storage resources using NVMe/TCP. Therefore, we proposed the zero-copy mechanism for the NVMe/TCP to optimize the I/O path for local access.

Essential service availability. The hot-upgrade and hot-migration of LightPool are essential to provide high available storage services. Users concern about storage service availability as much as I/O performance and capacity. In traditional ways, the upgrade and data migration usually result in storage services interruption. Therefore, we designed the hot-upgrade and hot-migration features in LightPool, which is transparent to users. They can accept a little performance fluctuations during hot-upgrade and hot-migrations for high availability.

Advantages of user-level storage engine. We design and implement LightPool-Engine in user-level for ease of development and high performance. User-level components are simpler to develop, operate, and maintain than those in kernel. We easily add hot-upgrade and monitoring features, which can hardly be implemented in kernel. We also deploy the QLC-based NVMe SSD to further reduce TCO and apply optimizations to achieve performance close to TLC-based disks. In addition, user-level components can be more easily integrated into the Kubernetes and run in pods for flexible

scheduling and deployment. For performance, user-level components can directly access storage devices, which is more efficient, reducing the overhead of kernel-user switch.

Scale of shared storage on each node. In our deployment, storage resources on a single node will not be allocated to more than other five nodes, which can balance effective utilization enhancement and the blast radius. Based on our experience, storage resources on one node can be fully utilized when are shared to less than five nodes. Therefore, once one node who provides storage fails, the impact can be minimize.

Benefits from emerging hardware. LightPool will benefit from emerging hardware, including high-performance network and DPU, which can further enhance the LightPool performance and reduce host resources consumption. High-performance network (such as RDMA, Alibaba Cloud eRDMA [6], AWS SRD [35], etc.) can further enhance the performance of LightPool. Meanwhile, many DPUs (such as NVIDIA BlueField-3 and Intel IPU [5]) have implemented the NVMe-oF engine, which we can easily utilize to enhance LightPool performance and release host resources.

VII. RELATED WORK

Cloud native storage. Many previous works [27]–[29] analyzed the performance and components of cloud-native storage in Kubernetes. And the work [37] analyzed the performance of containerized applications on the local and remote storage. CNSBench [28] is proposed to provide a cloud-native storage benchmark that contains control operations and allows easily combining traditional storage benchmark workloads. Kuberstorage [24] is a cloud-native storage engine for massive small files in Kubernetes architecture, enabling automatic disaster recovery, fault-tolerance, and efficient ca-

capacity expansion. To enable cloud-native applications to access various storage solutions transparently, USL [8] introduced an abstraction layer over Kubernetes storage. Many storage solutions are proposed for cloud scenario. Spool [38] proposed a reliable NVMe storage pool in public cloud infrastructure, which diagnoses the device failure type and ensures the consistency and correctness of the data. To fully benefits from emerging hardware technologies, BM-Store [10] is proposed as a FPGA-based local storage virtualization scheme for bare-metal clouds. Pangu [23], a large-scale distributed storage system in Alibaba, provides append-only file semantics like HDFS and multiple businesses are built on it.

Remote storage protocols. Many protocols are proposed to enable access to remote storage devices over the network, such as iSCSI and NVMe-oF [31]. There are many previous works that focus on iSCSI performance tuning [19], [36]. However, iSCSI cannot fully exploit the inherent parallelism and multiple queues of the emerging NVMe devices. In particular, iSCSI suffers from up to 40% performance degradation and consumes 30% more CPU resources for additional operations such as protocol conversion when accessing NVMe SSDs [17]. The previous work [18] also conducted experiments to understand the performance behaviors of NVMe-oF on Arm-based platforms. To further enhance the performance of NVMe-oF, previous work [20] proposed an adaptive communication channel to eliminate the high latency and low bandwidth. Many user-level NVMe-oF stacks are proposed to improve the performance, and CPU efficiency, including SPDK NVMe-oF target [40] and uNVMe-TCP [41]. To fully utilize the NVMe-oF offloading capability of emerging smart-NIC and DPUs, HyQ [9] adopted a hybrid I/O queue architecture, achieving both high performance and offloading advantages. These schemes can be adopted in both HCI [33] and storage disaggregation architecture [17], [21]. We choose the NVMe-oF in LightPool to achieve high performance.

VIII. CONCLUSION

In this paper, we design and implement LightPool, a novel high-performance storage pool architecture for cloud-native distributed databases, enhancing OceanBase cluster resource utilization from about 40% to 65%. LightPool enables the integration of storage disks in computing clusters into storage pools through NVMe-oF, with unified management and allocation for storage resources. LightPool has been deployed on more than 8500 nodes of the OceanBase cluster in Alibaba cloud. The deployment results demonstrate that LightPool achieves near-native performance of local disks. We are working on open-sourcing the LightPool and contributing to the community. The components on the control path of LightPool have been open-sourced and are available at <https://github.com/eosphoros-ai/liteio>.

IX. ACKNOWLEDGMENT

This research was supported by Alibaba Group through Alibaba Innovative Research Program. Wenzhi Chen is the corresponding author.

REFERENCES

- [1] "Kubernetes," <https://kubernetes.io/>, 2022.
- [2] "Flexible i/o tester," <https://github.com/axboe/fio.git>, 2021.
- [3] T. A. "Storage and pci express a natural combination," <http://www.avagotech.com/applications/datacenters/enterprise-storage>, 2016.
- [4] AWS., "Amazon elastic block store (amazon ebs)," <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/AmazonEBS.html>, 2023.
- [5] B. Bures, D. Daly, M. Debbage, E. Louzoun, C. Severns-Williams, N. Sundar, N. Turbovich, B. Wolford, and Y. Li, "Intel's hyperscale-ready infrastructure processing unit (ipu)," in *2021 IEEE Hot Chips 33 Symposium (HCS)*. IEEE, 2021, pp. 1–16.
- [6] H. Cao, C. Xu, Y. Han, M. Lin, K. Shen, G. Wang, J. Li, X. Sun, R. He, L. You *et al.*, "An efficient cloud-based elastic rdma protocol for hpc applications," *CCF Transactions on High Performance Computing*, pp. 1–9, 2023.
- [7] W. Cao, F. Li, G. Huang, J. Lou, J. Zhao, D. He, M. Sun, Y. Zhang, S. Wang, X. Wu, H. Liao, Z. Chen, X. Fang, M. Chen, C. Liang, Y. Luo, H. Wang, S. Wang, Z. Ma, X. Yang, X. Peng, Y. Ruan, Y. Wang, J. Zhou, J. Wang, Q. Hu, and J. Kang, "Polardb-x: An elastic distributed relational database for cloud-native applications," in *2022 IEEE 38th International Conference on Data Engineering (ICDE)*, 2022, pp. 2859–2872.
- [8] A. Chazapis, C. Pinto, Y. Gkoufas, C. Kozanitis, and A. Bilas, "A unified storage layer for supporting distributed workflows in kubernetes," in *Proceedings of the Workshop on Challenges and Opportunities of Efficient and Performant Storage Systems*, 2021, pp. 1–9.
- [9] Y. Chen, J. Chen, Y. Wang, Y. Chen, Z. Jin, J. Xu, G. Fang, W. Lin, C. Wei, and W. Chen, "Hyq: Hybrid i/o queue architecture for nvme over fabrics to enable high-performance hardware offloading," in *2023 IEEE/ACM 23rd International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*. IEEE, 2023, pp. 13–24.
- [10] Y. Chen, J. Xu, C. Wei, Y. Wang, X. Yuan, Y. Zhang, X. Yu, Y. Chen, Z. Wang, S. He *et al.*, "Bm-store: A transparent and high-performance local storage architecture for bare-metal clouds enabling large-scale deployment," in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2023, pp. 1031–1044.
- [11] A. Cloud., "Alibaba cloud enhanced ssds (essds)," <https://www.alibabacloud.com/help/en/elastic-compute-service/latest/essds>, 2022.
- [12] A. Cloud, "Polardb-x: Designed to address database challenges such as ultra-high concurrency, massive data storage, and large table performance bottlenecks," <https://www.alibabacloud.com/product/polardb-x>, 2023.
- [13] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears, "Benchmarking cloud serving systems with ycsb," in *Proceedings of the 1st ACM symposium on Cloud computing*, 2010, pp. 143–154.
- [14] S. Dong, A. Kryczka, Y. Jin, and M. Stumm, "Rocksdb: Evolution of development priorities in a key-value store serving large-scale applications," *ACM Transactions on Storage (TOS)*, vol. 17, no. 4, pp. 1–32, 2021.
- [15] S. Dong, S. S. P. S. Pan, A. Ananthabhotla, D. Ekambaram, A. Sharma, S. Dayal, N. V. Parikh, Y. Jin, A. Kim *et al.*, "Disaggregating rocksdb: A production experience," *Proceedings of the ACM on Management of Data*, vol. 1, no. 2, pp. 1–24, 2023.
- [16] T. A. S. Foundation., "Cassandra: Open source nosql database," <https://cassandra.apache.org/>, 2023.
- [17] Z. Guz, H. Li, A. Shayesteh, and V. Balakrishnan, "Nvme-over-fabrics performance characterization and the path to low-overhead flash disaggregation," in *Proceedings of the 10th ACM International Systems and Storage Conference*, 2017, pp. 1–9.
- [18] Y. Jia, E. Anger, and F. Chen, "When nvme over fabrics meets arm: performance and implications," in *2019 35th Symposium on Mass Storage Systems and Technologies (MSST)*. IEEE, 2019, pp. 134–140.
- [19] A. Joglekar, M. E. Kounavis, and F. L. Berry, "A scalable and high performance software iscsi implementation," in *FAST*, vol. 5, 2005, pp. 20–20.

- [20] A. Kashyap and X. Lu, "Nvme-oaf: Towards adaptive nvme-of for io-intensive workloads on hpc cloud," in *Proceedings of the 31st International Symposium on High-Performance Parallel and Distributed Computing*, 2022, pp. 56–70.
- [21] A. Klimovic, C. Kozyrakis, E. Thereska, B. John, and S. Kumar, "Flash storage disaggregation," in *Proceedings of the Eleventh European Conference on Computer Systems*, 2016, pp. 1–15.
- [22] A. Klimovic, H. Litz, and C. Kozyrakis, "Reflex: Remote flash = local flash," *ACM SIGARCH Computer Architecture News*, vol. 45, no. 1, pp. 345–359, 2017.
- [23] Q. Li, Q. Xiang, Y. Wang, H. Song, R. Wen, W. Yao, Y. Dong, S. Zhao, S. Huang, Z. Zhu *et al.*, "More than capacity: Performance-oriented evolution of pangu in alibaba," in *21st USENIX Conference on File and Storage Technologies (FAST 23)*, 2023, pp. 331–346.
- [24] F. Liu, J. Li, Y. Wang, and L. Li, "Kubestorage: A cloud native storage engine for massive small files," in *2019 6th International Conference on Behavioral, Economic and Socio-Cultural Computing (BESCC)*. IEEE, 2019, pp. 1–4.
- [25] Mayastor, "Mayastor user referenc: Deploy a test application." <https://mayastor.gitbook.io/introduction/quickstart/deploy-a-test-application>, 2023.
- [26] Mayastor, "The native nvme-of cas engine of openebs." <https://mayastor.gitbook.io/>, 2022.
- [27] L. Mercl and J. Pavlik, "Public cloud kubernetes storage performance analysis," in *International Conference on Computational Collective Intelligence*. Springer, 2019, pp. 649–660.
- [28] A. Merenstein, V. Tarasov, A. Anwar, D. Bhagwat, J. Lee, L. Rupprecht, D. Skourtis, Y. Yang, and E. Zadok, "{CNSBench}: A cloud native storage benchmark," in *19th USENIX Conference on File and Storage Technologies (FAST 21)*, 2021, pp. 263–276.
- [29] A. Merenstein, V. Tarasov, A. Anwar, D. Bhagwat, L. Rupprecht, D. Skourtis, and E. Zadok, "The case for benchmarking control operations in cloud native storage," in *12th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage 20)*, 2020.
- [30] I. MongoDB, "Mongodb: For the next generation of intelligent applications." <https://www.mongodb.com/>, 2023.
- [31] NVMEEXPRESS, "Nvm express specification," <http://www.nvmexpress.org/specifications/>, 2011.
- [32] OceanBase., "An open source, ultra-fast & cost-effective distributed sql database," <https://en.oceanbase.com/>, 2023.
- [33] OpenEBS, <https://openebs.io/>, 2022.
- [34] J. Ouyang, S. Lin, S. Jiang, Z. Hou, Y. Wang, and Y. Wang, "Sdf: Software-defined flash for web-scale internet storage systems," in *Proceedings of the 19th international conference on Architectural support for programming languages and operating systems*, 2014, pp. 471–484.
- [35] L. Shalev, H. Ayoub, N. Bshara, and E. Sabbag, "A cloud-optimized transport protocol for elastic and scalable hpc," *IEEE Micro*, vol. 40, no. 6, pp. 67–73, 2020.
- [36] D. Xinidis, A. Bilas, and M. D. Flouris, "Performance evaluation of commodity iscsi-based storage systems," in *22nd IEEE/13th NASA Goddard Conference on Mass Storage Systems and Technologies (MSST'05)*. IEEE, 2005, pp. 261–269.
- [37] Q. Xu, M. Awasthi, K. T. Malladi, J. Bhimani, J. Yang, and M. Annavaram, "Performance analysis of containerized applications on local and remote storage," in *Proc. of MSST*, vol. 3, 2017, pp. 24–28.
- [38] S. Xue, S. Zhao, Q. Chen, G. Deng, Z. Liu, J. Zhang, Z. Song, T. Ma, Y. Yang, Y. Zhou *et al.*, "Spool: Reliable virtualized nvme storage pool in public cloud infrastructure," in *2020 USENIX Annual Technical Conference (USENIX ATC 20)*, 2020, pp. 97–110.
- [39] Z. Yang, C. Yang, F. Han, M. Zhuang, B. Yang, Z. Yang, X. Cheng, Y. Zhao, W. Shi, H. Xi *et al.*, "Oceanbase: a 707 million tpmc distributed relational database system," *Proceedings of the VLDB Endowment*, vol. 15, no. 12, pp. 3385–3397, 2022.
- [40] Z. Yang, C. Liu, Y. Zhou, X. Liu, and G. Cao, "Spdk vhost-nvme: accelerating i/os in virtual machines on nvme ssds via user space vhost target," in *2018 IEEE 8th International Symposium on Cloud and Service Computing (SC2)*. IEEE, 2018, pp. 67–76.
- [41] Z. Yang, Q. Wan, G. Cao, and K. Latecki, "unvme-tcp: a user space approach to optimizing nvme over fabrics tcp transport," in *International Conference on Internet of Vehicles*. Springer, 2020, pp. 125–142.